Case Study Submission – Integration Microservices Engineer – Stephen Muema

Candidate	Stephen Muema
Role	Integration Microservices Engineer
Submission date	10 June 2026
GitHub repository	https://github.com/stephenmuema/loop-integration-microservices-assessment

1. Overview

This submission implements a production-style **Spring Boot integration microservice** that accepts a country name over REST, resolves it through the public **CountryInfo SOAP service** (two chained SOAP operations), persists the result to **MySQL**, and exposes full **CRUD** over the stored data. The service is fully containerised with Docker and deployable to **Kubernetes**, with resilience, observability, structured error handling, and security baked in.

End-to-end flow

```
POST /api/countries {"name":"kenya"}
   └─ sentence-case  -> "Kenya"
      └─ SOAP CountryISOCode("Kenya")     -> "KE"
         └─ SOAP FullCountryInfo("KE")    -> {name, capital, phone, continent,
                                              currency, flag, languages[]}
            └─ upsert (by ISO code) -> MySQL
               └─ 201 Created (CountryResponse with opaque UUID id)
```

2. How the requirements were met

Core tasks

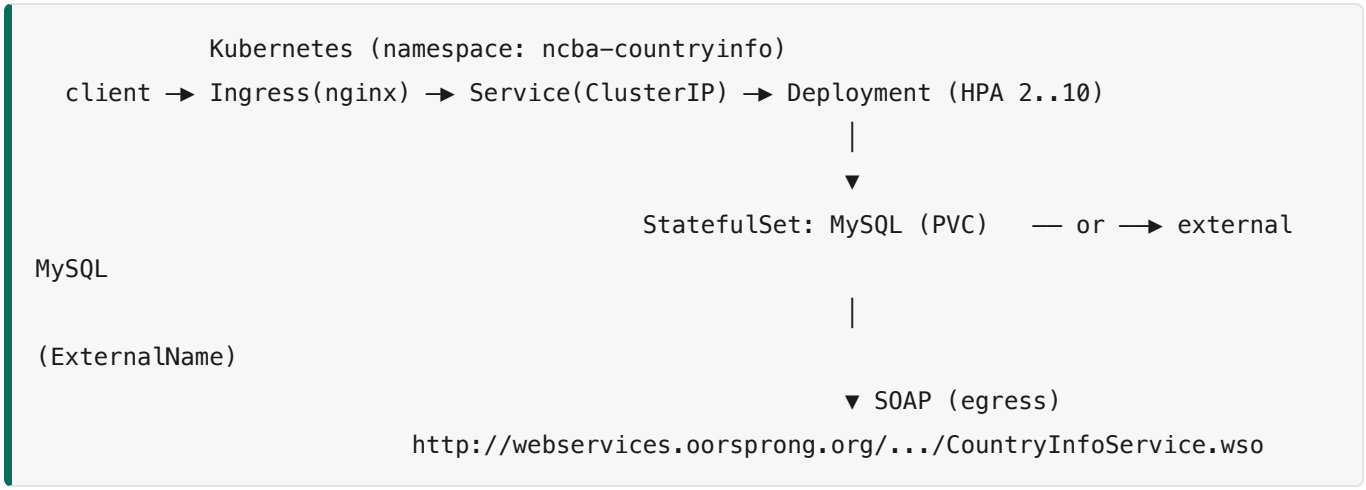
#	Requirement	Implementation
1	Spring Boot app (Web, JPA, MySQL)	Spring Boot 3.2, Java 17, Maven
2	Consume the CountryInfo SOAP WSDL	Spring-WS `WebServiceTemplate` + JAXB bindings
3	POST endpoint receiving `{name}`, sentence-cased	`POST /api/countries`, `StringUtils.toSentenceCase`
4	Call SOAP for ISO code (`sCountryName`)	`SoapCountryClient.getIsoCode`
5	Use ISO code to fetch `FullCountryInfo`	`SoapCountryClient.getFullCountryInfo`
6	`CountryInfo` + `Language` models	JPA entities, one-to-many, upsert by ISO code
7	CRUD REST APIs (all, by id, update, delete)	`CountryController` — GET/GET{id}/PUT/DELETE
8	Kubernetes deployment scripts	`k8s/` manifests (namespace, config, secret, app, db, svc, ingress, HPA)
9	Deployment guide	`docs/kubernetes-deployment-guide.(md|pdf)`
10	Troubleshooting guide	`docs/kubernetes-troubleshooting-guide.(md|pdf)`

“Important Notes” (non-functional)

Requirement	How it is satisfied	Status
Clear system design + justified trade-offs	Architecture diagram + design-decision table in README	✅
High load: stateless, horizontal scaling, load balancing	Stateless pods, K8s `Service` load-balancing 2 replicas, HPA (2→10 @70% CPU)	✅ (LB proven; HPA configured)
Failure handling: retries, timeouts, circuit breakers, fallbacks	Resilience4j retry (3×, exp. backoff), 5s/10s timeouts, circuit breaker + fallback → `502`	✅ Proven live
Structured logging, metrics, monitoring	JSON logs (logback), Micrometer + `/actuator/prometheus`, health with DB/SOAP/breaker	✅ Proven
Separation of concerns (MVC)	controller / service / repository / model / dto / client / config / exception / util	✅
Robust error handling + proper status codes	`@RestControllerAdvice`, structured error envelope, 400/404/502/500	✅ Proven
Production-ready deployment	Multi-stage non-root image, ConfigMap/Secret, readiness/liveness probes	✅ Proven
Steps to run and test	README §4 (4 run options), §5 tests, Postman collection, docs	✅

---

3. Architecture & key design decisions



Decision	Rationale / trade-off
Spring-WS `WebServiceTemplate` + JAXB	Strongly-typed, marshalled SOAP payloads instead of brittle hand-built XML. JAXB classes are hand-written, so the build needs no code-generation step.
Resilience4j for retry + circuit breaker + fallback	One unified, annotation-driven, observable resilience model (Micrometer + Actuator health) — the maintained successor to Hystrix. Composes more cleanly than mixing Spring Retry with a separate breaker.
Upsert keyed on ISO code	ISO code is the natural key (unique column); re-ingest updates rather than duplicating — idempotent.
Opaque UUID `publicId` as the only API identifier	Mitigates IDOR / record enumeration. The sequential primary key stays internal for indexing; the public id is random and non-guessable. Trade-off: one extra unique-indexed column.
Stateless services + externalised config	Every replica is interchangeable → horizontal scaling behind the Service/HPA. All config/secrets via env vars (12-factor), same image across environments.
External SOAP excluded from readiness probe	A transient upstream outage should not eject pods that can still serve persisted CRUD reads; SOAP health remains visible for alerting.

4. Resilience (verified live)

The SOAP client is wrapped with Resilience4j **retry** (3 attempts, exponential backoff), explicit **connect/read timeouts** (5s/10s), and a **circuit breaker** with a **fallback** that converts failures into a clean `502 Bad Gateway`.

The breaker was demonstrated end-to-end by pointing the SOAP endpoint at a dead address and driving traffic:

```
CLOSED  -> request fails through retries -> 502
OPEN    -> failure-rate threshold crossed -> calls blocked from the downstream
HALF_OPEN -> after the 10s wait, probes for recovery
CLOSED  -> endpoint restored -> 201 OK, breaker resets
```

State is observable at `/actuator/health` and via the `resilience4j_circuitbreaker_state` Prometheus metric.

---

5. Observability & error handling

- **Structured JSON logs** (logback + logstash encoder under the `prod` profile); entry/exit logging on every service and client method.
- **Metrics**: Micrometer with `/actuator/metrics` and `/actuator/prometheus` (HTTP server metrics + Resilience4j metrics); Prometheus scrape annotations on the Deployment.
- **Health**: `/actuator/health` aggregates DB, SOAP reachability, and circuit breaker; dedicated `/health/readiness` and `/health/liveness` for Kubernetes.
- **Error handling**: a `@RestControllerAdvice` returns a consistent structured envelope (`timestamp, status, error, message, path`, plus `fieldErrors` for validation) with correct codes — `400` (validation), `404` (not found), `502` (SOAP failure), `500` (safe generic) — never leaking stack traces.

---

6. Deployment (verified on Kubernetes)

- **Docker**: multi-stage build (Maven → JRE-Alpine), non-root user, JVM container tuning. `docker-compose.yml` runs app + MySQL locally with a health-gated database.
- **Kubernetes** (`k8s/`): namespace, ConfigMap, Secret, MySQL StatefulSet + PVC, app Deployment (2 replicas, resource requests/limits, readiness/liveness probes, zero-downtime rolling updates), ClusterIP Service, NGINX Ingress, and an HPA (2→10 @70% CPU).
- **External-database mode**: `k8s/external-mysql.yaml` swaps the in-cluster DB for a managed/standalone MySQL via an `ExternalName` Service — no app changes.

Verified on Docker Desktop Kubernetes: MySQL StatefulSet bound its PVC, the app rolled out 2/2 with passing probes, the Service load-balanced both pods, and the full CRUD flow worked end-to-end through the Service (live SOAP → MySQL). The external-MySQL mode was also verified by pointing the app at a host MySQL.

---

7. Testing & quality

- **42 tests** (JUnit 5, Mockito, MockMvc): unit tests for the service, SOAP client mapping/fallbacks, sentence-case utility, and exception handler; an end-to-end web-layer integration test covering the full POST flow and all CRUD
    - error cases.
- **JaCoCo coverage gate at 80%** (build fails below it); achieved **97% line / 90% branch** on the business code.
- **Postman collection** (`postman/`) with chained requests and assertions, including an IDOR-probe test.

---

8. How to run & test (quick start)

```
git clone https://github.com/stephenmuema/loop-integration-microservices-assessment
cd loop-integration-microservices-assessment
cp .env.example .env
docker compose up --build -d           # starts MySQL + app

curl http://localhost:8080/actuator/health
curl -X POST http://localhost:8080/api/countries \
  -H 'Content-Type: application/json' -d '{"name":"kenya"}'
```

Full instructions (Maven, runnable jar, Kubernetes, external MySQL), the API reference, and the deploy/troubleshooting guides are in the repository `README` and `docs/` folder.

---

9. Repository contents

```
├── src/                          Spring Boot application (MVC layers) + tests
├── k8s/                          Kubernetes manifests (+ external-mysql.yaml)
├── docs/                         API docs, K8s deploy & troubleshooting guides (md + pdf)
├── postman/                      Postman collection + environment
├── Dockerfile                    Multi-stage, non-root image
├── docker-compose.yml            Local app + MySQL stack
└── README.md                     Architecture, design decisions, run/test/deploy guides
```

Repository: https://github.com/stephenmuema/loop-integration-microservices-assessment